

National University of Computer and Emerging Sciences

Practical Project

Near-Real-Time and Periodic Data Warehouse with MESHJOIN

NAME:	Dawar Ahmed
ROLL NUMBER:	24I - 2533
DEGREE PROGRAM:	BS DS
SECTION:	5B
SUBJECT NAME:	Data Warehousing and Business Intelligence
DATE OF SUBMISSION:	13 – 11 – 26
SUBMITTED TO:	**Dr. Asif Naeem**

Building and Analysing a Near-Real-Time Data Warehouse for a Retail Chain with MESHJOIN

1. Project overview

The project builds a near-real-time data warehouse (DW) for the sales of a UK online gift retailer. Sales arrive in two forms. **Online** sales come in one by one as a stream and must reach the DW within seconds. **Periodic** (store) sales are collected and arrive once every simulated hour in groups. Each sale carries only a customer id and a product id, so before it can be loaded it must be joined with the customer and product **Master Data**, which is kept on disk.

The join is done with **MESHJOIN** (Mesh Join), a stream-to-disk join that reads the Master Data one partition at a time in a loop and matches it against a window of the stream held in memory. When the stream arrives faster than MESHJOIN can process it (peak time), sales are **shed** to a disk table and processed later in non-peak time, with online sales always having priority.

The work was done in this order:

1. Clean the transactional file (4 rules) and split it into online and periodic sales with an MD5 rule.
2. Build the customer and product Master Data, the online stream file and 48 hourly periodic groups.
3. Create a star schema in MySQL (5 dimensions, 1 fact table).
4. Implement MESHJOIN in Python with three threads (OnlineThread, PeriodicThread, MeshJoinThread), load shedding and live commands, and load the joined data into the DW.
5. Analyse the DW with 20 OLAP queries, a view and a materialized view.

The final load put all **397,884** cleaned transactions into the DW, with revenue **8,911,407.904** equal to the sum of quantity x unit price over the cleaned data. All code is in Python 3 and MySQL 8.0.

2. Dataset and cleaning

2.1 Source

Chen, D., Sain, S. L., and Guo, K. **Online Retail.** UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/352/online+retail. Downloaded on **3 November 2026** (file `Online Retail.xlsx`).

The file has 541,909 transaction lines from 1 December 2010 to 9 December 2011 with the columns InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID and Country. One invoice (InvoiceNo) is one shopping cart.

2.2 Cleaning rules and counts

Only the four rules given in the project statement were applied. Duplicate lines and non-product stock codes were kept, because the statement does not ask for their removal. IDs were read as text so that codes such as 85123A keep their letters; CustomerID was written without a decimal part (17850, not 17850.0).

Table 1. Cleaning counts. The program applies the four rules together; the order only matters for the breakdown.

Rule (counted in this order)	Removed	Rows left
Start		541,909
CustomerID must be present	135,080	406,829
InvoiceNo must not start with C (cancellation)	8,905	397,924
Quantity > 0	0	397,924
UnitPrice > 0	40	397,884

Each rule on its own flags 135,080 rows without a customer, 9,288 cancellation lines, 10,624 lines with quantity <= 0 and 2,517 lines with price <= 0. The rules overlap: every line with a negative quantity is either a cancellation or has no customer, so after the first two rules none is left. After cleaning: **397,884** lines, 18,532 carts, 4,338 customers, 3,665 products, 37 countries.

3. Preparing the data

3.1 The sales_mode rule

Every cart is labelled Online or Periodic with a deterministic MD5 rule, applied to the InvoiceNo so that all lines of one cart get the same label:

```
n = int(first 8 hex digits of MD5(transaction_id), 16)
n mod 10 in 0..6  ->  Online      n mod 10 in 7..9  ->  Periodic
```

Example: MD5("536365") starts with 3e6d49ce, so n = 1,047,349,710 and n mod 10 = 0: cart 536365 is **Online**. Result: 12,971 online carts (69.99%, 278,740 rows) and 5,561 periodic carts (30.01%, 119,144 rows). No cart is split between the two forms.

3.2 The store rule

Online sales belong to store 1 (Online, Web). A periodic cart goes to store 2 + (n mod 4), using the same n. The store list (5 stores) is kept in memory and joined outside MESHJOIN. Because periodic carts have n mod 10 equal to 7, 8 or 9, two of the three cases are odd numbers, so stores 3 and 5 receive about twice as many carts as stores 2 and 4 (909, 1,869, 920 and 1,863 carts). I kept the formula exactly as given and only report the effect.

3.3 Master Data

Customers (4,338) and products (3,665) are taken from the cleaned data. Their missing attributes are generated with `stable(text, n) = MD5 number of text mod n`, where the text is the id or the id plus a suffix for that attribute (for example "17850age" for the age group), so the attributes do not depend on each other and the same id always gets the same value. For example customer 17850 is Female, 51-55, Engineer, city category A, 1 year in the city, Married.

- **Customer:** gender, age group, occupation, city category, years in current city, marital status (city category from the id plus the customer's country; 8 customers with two countries take the country of their first cleaned transaction).
- **Product:** name (the longest description; 54 ties broken by file order), category, supplier id and supplier name (5 suppliers).

Both files are sorted by id and stored in MySQL with a `partition_no = row number div 500`, giving **9 customer partitions and 8 product partitions** of 500 rows. MESHJOIN reads one partition of each per iteration with `WHERE partition_no = k` on an index.

3.4 Working data

The online sales are written to `online_sales_stream.csv` (278,740 rows, original time order). The periodic sales are sorted by time and loaded into the staging table `periodic_sales_data` in **48 groups**, one per simulated hour. Whole carts are kept together: a cart goes to group (start row x 48) div 119,144. The groups hold **2,263 to 2,710** rows and no cart is split (an equal cut would have split 44 carts).

4. DW schema

The DW is a **star schema** with one fact table and five dimensions (Figure 1, on the page after this section). The grain of the fact table is **one transaction line** (one product in one cart), so every cleaned row becomes exactly one fact row.

Table 2. Tables of the DW.

Table	Contents	Rows loaded
dim_customer	customer_id and the six generated attributes	4,338
dim_product	product_id, product_name, product_category	3,665
dim_supplier	supplier_id, supplier_name	5
dim_store	store_id, store_name, city	5
dim_time	one row per minute: date, hour, day of month, month, quarter, year, day of week, weekday/weekend, season, half-year	17,282
fact_sales	five foreign keys, transaction_id, quantity, revenue, sales_mode, processing_mode	397,884

- Every dimension has an integer **surrogate key** (AUTO_INCREMENT); the original ids are kept as attributes. dim_time uses `time_key = YYYYMMDDHHMM` (for example 201012010826), which is unique per minute and sorts in time order.
- **Supplier** is its own dimension so that supplier questions (Q13, Q15, Q17) can group by it directly.
- All time parts are stored as columns, so no query has to cut up a timestamp. Spring is March to May, summer June to August, fall September to November, winter December to February; the first half-year is January to June.
- **revenue = quantity x unit price** is computed during loading and stored as DECIMAL(14,3); unit_price uses three decimals because some prices are 0.001.
- `sales_mode` (Online / Periodic) says where the sale happened; `processing_mode` (Live / Periodic / Shed) says how the ETL handled it. CHECK constraints allow only these values.
- An index on fact_sales.transaction_id supports the shopping-cart query (Q16).

`Create-DW.sql` first drops the view STORE_QUARTERLY_SALES, the materialized-view table and all tables, then creates the working tables (Master Data, staging, shed table) and the star schema with its primary and foreign keys (InnoDB, utf8mb4).

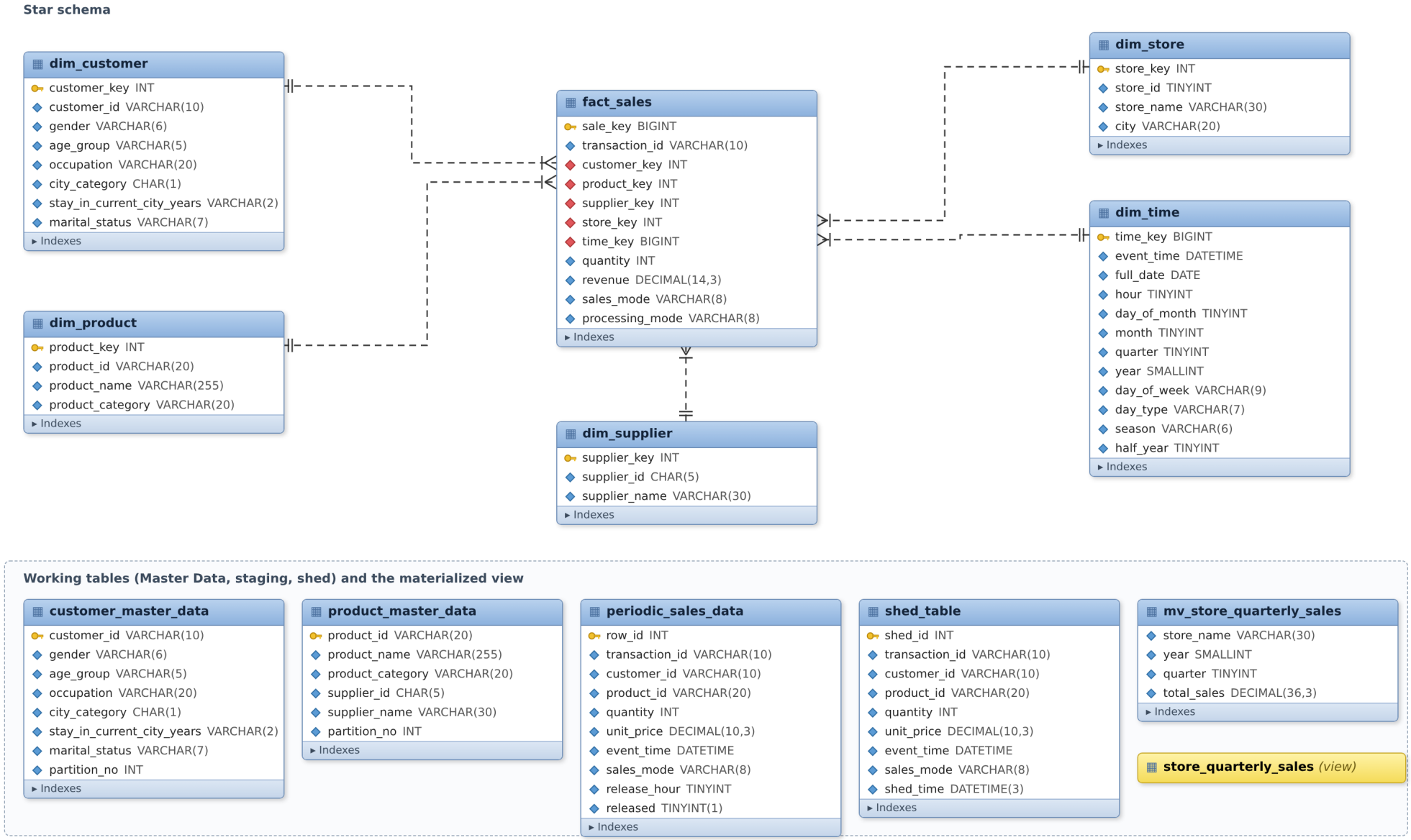


Figure 1. DW schema: the star schema (top) and the working tables (bottom). Key = primary key, red diamond = foreign key, dashed lines = one-to-many relationships.

5. MESHJOIN

5.1 How MESHJOIN works

MESHJOIN joins a fast stream S with a large disk relation R without looking up R once per stream tuple, which would cost one random disk access per tuple. Its parts are:

- **Stream buffer:** incoming tuples wait here (limit 1,000).
- **Hash table H:** the stream tuples currently being joined, indexed by the join key. Several tuples can share one key.
- **Queue:** pointers to the chunks in H in arrival order (each chunk is w = 500 tuples).
- **Disk buffer:** one partition of R (500 rows).

In every iteration MESHJOIN (1) moves up to w tuples from the stream buffer into H and the queue, (2) reads the next partition of R into the disk buffer, (3) looks up every row of that partition in H and joins the matching tuples, and (4) removes the oldest chunk once it has been in H for a full cycle, that is, after it has met every partition of R. R is read in an endless loop, so each disk read serves all the chunks in memory at once.

5.2 MESHJOIN with two Master Data tables

Each transaction needs both a customer and a product. The instructor asked not to clone the master tables and not to run MESHJOIN twice, because disk access is the expensive part. I therefore scan **both tables together**: every iteration reads one customer partition and one product partition, and H has two indexes (by customer_id and by product_id) over the same tuples.

- A chunk stays for **cycle = max(9, 8) = 9 iterations**. In that time it meets all 9 customer partitions once and all 8 product partitions, one of them twice.
- Each tuple has `customer_matched` and `product_matched` flags, so the product page that comes twice does not enrich it again.
- The alternatives were two MESHJOINs in a row (17 iterations per chunk, two hash tables) or one long cycle over both tables (17 iterations, twice the memory). Scanning together gives the shortest wait and the least memory for the same 2 disk reads per iteration.

Key lines of one iteration:

```
cust_page = db.read_partition(conn, "customer_master_data", self.k_cust)
prod_page = db.read_partition(conn, "product_master_data", self.k_prod)
for row in cust_page:
    for t in self.H_cust.get(row["customer_id"], []):
        if not t["customer_matched"]:  ... copy the 6 attributes ...
self.k_cust = (self.k_cust + 1) % self.n_cust      # endless loop over R
if self.queue and self.queue[0][1] >= self.cycle:   # oldest chunk leaves
```

5.3 Threads and data flow

Figure 2 shows the whole program. Three threads feed and run MESHJOIN, and a loader thread writes the result to the DW:

- **OnlineThread** reads `online_sales_stream.csv` and puts sales into the stream buffer at 80 per second in peak time and 10 per second in non-peak time.
- **PeriodicThread** releases one group from the staging table every simulated hour (60 seconds), marks it released and puts its carts into the buffer.
- **MeshJoinThread** takes up to 500 sales per iteration, **online first, then periodic, then shed**, runs the iteration and passes finished sales on, adding the store from the in-memory store list.
- **LoaderThread** computes revenue and loads the DW (Section 7). The main thread reads the user's commands.

The stream buffer has two lines (online and periodic) protected by one lock (threading.Condition); a thread that finds the buffer full waits until MESHJOIN takes sales out. Each thread that uses MySQL has its own connection.

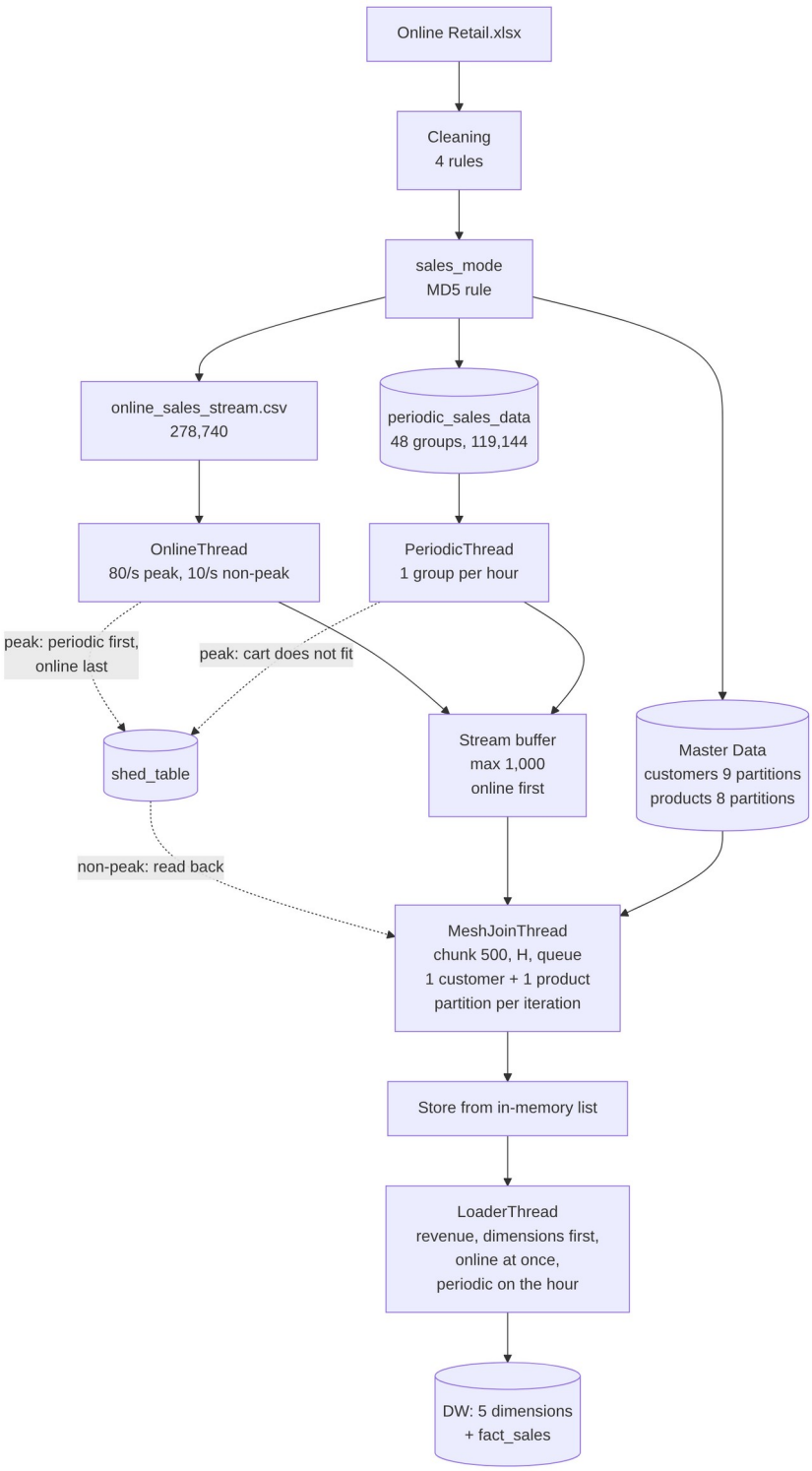


Figure 2. Data flow of the program. Dotted arrows are used only by load shedding.

Check on the real data: all 397,884 sales came out of MESHJOIN with both customer and product attributes, 0 were left without a match, and a test on 3,000 sales gave 0 differences from a direct lookup in the master tables.

6. Load shedding and commands

6.1 Rules

In **peak** time the program sheds instead of waiting when the buffer is full. Following the instructor's answer ("always shed periodic first, then online") room is made in this order:

1. An arriving periodic cart that does not fit in the buffer is shed whole.
2. When an online sale arrives at a full buffer, the newest periodic cart waiting in the buffer is moved to the shed table to make room.
3. Only when the buffer holds online sales only is the online sale itself shed.

Shed sales go to `shed_table` with their flag (sales_mode) and the time they were shed. Periodic carts are always moved whole, so one shopping trip is never split between MESHJOIN and the shed table; online sales are shed one at a time because they arrive one at a time. In **non-peak** time nothing is shed; the threads wait for space, and MESHJOIN fills every chunk that has fewer than 500 sales from the shed table (online-flagged rows first), deletes what it read and marks those sales `processing_mode = Shed`. This gives the required priority: online, then periodic, then shed.

6.2 Commands

Table 3. Commands typed while the program runs.

Command	Effect
online / periodic / both	Run only the online stream, only the hourly groups, or both (online first)
peak / non-peak	Fast speed with shedding allowed / slow speed with shed sales processed
speed peak N, speed non-peak N	Change a speed while the program runs
status	Mode, speed, stream-buffer count, sales in H, sales still on the shed disk, shed and read-back counts, rows loaded
quit	Stop the threads, load what is left and close

6.3 Shed counts

Table 4. Shed counts. The final load is the run whose DW is analysed in Section 8.

Run	Shed in peak (online / periodic)	Shed in non-peak	Read back in non-peak
Final load of the DW (--peak 5000 --nonpeak 2000 --hour 2)	0 / 14,831	0	14,831
Earlier full run (same settings)	0 / 14,307	0	14,307
Stress test with a slow consumer (1,000 per second)	1,339 / 14,394	0	15,733

Every hourly group (2,263 to 2,710 sales) is larger than the 1,000-sale buffer, so in peak time part of each group is shed: about 12% of all periodic sales in both full runs. In the earlier run, 12,248 of the shed sales came in carts that did not fit when they arrived and 2,059 were taken out of the buffer to make room for online sales. No online sale was shed in the full runs, because MESHJOIN keeps up with the online stream alone; only the stress test, where the consumer was deliberately slowed, shed online sales, and only after the periodic ones. Nothing is ever shed in non-peak time by design. The exact numbers depend on how long the run stays in peak and on the speed of the computer.

7. Loading the DW

- **Consolidation:** revenue = quantity x unit price, computed with exact decimals (6 x 2.55 = 15.30), and the time row with all its parts.
- **Dimensions first:** the LoaderThread loads sales in batches (up to 1,000 or every 0.5 s). In each batch it first inserts every customer, product, supplier, store or minute that is not yet in the DW, then the fact rows with their five keys, and commits once. A small map of id to surrogate key tells it which values are already loaded; these are DW keys, not the Master Data, which MESHJOIN still reads only from disk.
- **Online at once, periodic on the hour:** online sales are loaded as soon as they are joined. Periodic sales (also those read back from the shed table) wait and are loaded together once every simulated hour, as the statement requires.
- **Bonus not used:** Live results are inserted directly into the fact table.

Table 5. Checks after the final load (test_dw.py computes the expected values from the online file and the staging table).

Check (expected from the working data)	Expected	In the DW
Fact rows	397,884	397,884
Total revenue	8,911,407.904	8,911,407.904
Customers / products	4,338 / 3,665	4,338 / 3,665
Suppliers / stores	5 / 5	5 / 5
Minutes in dim_time	17,282	17,282
Live / Periodic / Shed rows		278,740 / 104,313 / 14,831

8. DW analysis

`Queries-DW.sql` answers the 20 questions. The loaded dates are **1 December 2010 to 9 December 2011** (13 months), so "the loaded year" is **2011** and "the latest six months" are **July to December 2011** (December up to the 9th); the script names these dates in a comment and its first query prints them.

Table 6. How the queries slice, dice and drill down.

OLAP operation / SQL tool	Used in
Slice (one fixed value, year = 2011)	Q1, Q4, Q9, Q10, Q11, Q12, Q17, Q18
Dice (several dimensions at once)	Q2 gender x age x city category; Q6 six months x city category x marital status
Drill-down (year to quarter to month)	Q1, Q4, Q6, Q9, Q11
Roll-up with subtotals (WITH ROLLUP, GROUPING)	Q17
Top-N inside groups (ROW_NUMBER, RANK)	Q1, Q5, Q8, Q11
Growth and volatility (LAG)	Q9, Q12, Q15
View and materialized view	Q20

8.1 Main results

- **Products bought together (Q16):** the top pairs are the same product in two colours or designs: JUMBO BAG PINK POLKADOT + JUMBO BAG RED RETROSPOT (546 carts), ROSES + GREEN REGENCY TEACUP AND SAUCER (541), ALARM CLOCK

BAKELIKE GREEN + RED (530), and two LUNCH BAG pairs (523, 519). These are natural bundles.

- **Weekdays and weekends (Q1, Q10):** weekend sales are small for every age group, for example 0-17: 100,198 of 1,573,432 in 2011. The data contains Sunday sales but **no Saturday sales**.
- **Seasonality (Q4, Q12):** sales grow strongly towards the end of the year; Birmingham Bullring grew 117% from Q3 to Q4 2011 even though December stops on the 9th.
- **Biggest products (Q1, Q14, Q18):** the top weekday product of 2011 is PAPER CRAFT, LITTLE BIRDIE (168,469.60), entirely from one order of 80,995 units on 9 December 2011. Its cancellation invoice starts with C and was removed by the cleaning rule, while the original order is a valid row.

8.2 Revenue spikes (Q19)

Q19 returns 22,885 product-days on which a product earned more than twice its own average daily revenue (average over the days it sold). The largest spikes are single bulk orders:

Table 7. The five largest spikes.

Product	Day	Revenue that day	Daily average	Times average
MEDIUM CERAMIC TOP STORAGE JAR	2011-01-18	77,183.60	707.97	109.0
PICNIC BASKET WICKER 60 PIECES	2011-06-10	39,916.50	422.91	94.4
DOORSTOP RETROSPOT HEART	2011-09-20	3,035.52	37.11	81.8
METAL SIGN TAKE IT OR LEAVE IT	2011-12-07	3,861.00	49.29	78.3
PINK BLUE FELT CRAFT TRINKET BOX	2011-10-07	3,065.16	51.92	59.0

Comments on the spikes:

- The two largest are wholesale orders of a whole year's volume on one day (74,215 storage jars in January, a large order of picnic baskets in June). A product's average is pulled up by such a day, but it still stands out by about 100 times.
- Several of the next spikes fall in September to December, which matches the pre-Christmas buying by wholesale customers seen in Q12.
- PAPER CRAFT, LITTLE BIRDIE is the biggest single order but is **not** a spike: it sold on one day only, so that day is its average. The "twice the average" rule can only find spikes for products that sell on several days.
- Many of the 22,885 rows are small products with irregular sales, where one ordinary order is already more than twice a low average. A minimum revenue or a median-based average would give a shorter, more useful list.

8.3 View and materialized view

Q20 creates the view STORE_QUARTERLY_SALES (store, year, quarter, total sales: 5 stores x 5 quarters = 25 rows); the ORDER BY by store name and quarter is on the SELECT that reads the view, because MySQL does not keep it inside CREATE VIEW. MySQL has no materialized views, so the view's result is stored in the table `MV_STORE_QUARTERLY_SALES`, refreshed by dropping and creating it again after a load. On the laptop, reading the view took 4.0 s (the joins and sums run every time) and reading the stored table took 0.0 s.

The pair query Q16 needed tuning: the first version did not finish in reasonable time. Two observations made it fast: a pair can never be in more carts than either of its products, so only products bought in at least 400 carts are paired (the 5th pair is in 519 carts, so the result is exact),

and joining the fact table to itself on its transaction_id index instead of a temporary list. It now runs in 22.7 s.

9. Three shortcomings of MESHJOIN

1. Every tuple waits a full cycle. A chunk leaves H only after it has met every partition of the Master Data (9 iterations here), even when all its tuples were matched in the first iteration. With unique ids each sale matches exactly one customer and one product, so it could be emitted as soon as both flags are set. The fixed cycle adds delay to every sale and keeps finished tuples in memory, which fills H sooner and causes more shedding.

2. It does not use the skew of the data. MESHJOIN reads every partition equally often, but sales are very uneven: a few hundred popular products and frequent customers appear in most carts. The pages that hold them are read no more often than pages with rarely sold products, and nothing is cached. Later algorithms such as HYBRIDJOIN and CACHEJOIN keep frequently used Master Data in memory for this reason.

3. Its cost grows with the Master Data and its settings are fixed. The cycle length, and so the waiting time and the memory for H, grows with the number of partitions; with two master tables the cycle is the longer of the two. The chunk size, partition size and buffer size are fixed in advance and do not adapt to the arrival rate, so when the stream speeds up (an hourly group of about 2,500 sales against a 1,000-sale buffer) MESHJOIN cannot speed up with it and load shedding is needed.

10. What I learned

- **Streams and disks need different joins.** Looking up each sale on disk would be simple but slow; MESHJOIN shares one disk read among all the sales in memory. Choosing how to handle two master tables showed me the trade-off between disk reads, memory and waiting time.
- **Threads need care.** Sharing one buffer between four threads taught me to use a lock with wait and notify, to give each thread its own database connection, and to plan how every thread stops. One bug came from MySQL showing a connection an old snapshot of the shed table; turning on autocommit fixed the status count.
- **Small rules matter.** Keeping shopping carts whole needed checks in several places; a test on the real data found one cart split by the chunk boundary, which I fixed. Exact decimals for money, minute-level time keys and IDs read as text all prevented quiet errors.
- **Priorities under load.** Load shedding made it concrete that near-real-time means deciding what can wait: online sales first, store sales later, and nothing lost.
- **Check against independent numbers.** Each phase was tested against values computed separately (counts, revenue, a direct lookup), which caught problems early and makes me confident the 397,884 rows in the DW are right.
- **SQL performance.** The product-pairs query showed that the same answer can take minutes or seconds depending on how much data the query has to touch.

References

Chen, D., Sain, S. L., and Guo, K. Online Retail. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/352/online+retail (downloaded 3 November 2026).

Polyzotis, N., Skiadopoulos, S., Vassiliadis, P., Simitsis, A., and Frantzell, N.-E. Meshing Streaming Updates with Persistent Data in an Active Data Warehouse. IEEE Transactions on Knowledge and Data Engineering, 20(7), 2008.

Kimball, R., and Ross, M. The Data Warehouse Toolkit, 3rd edition. Wiley, 2013.